DEVOPS MINI PROJECT

Automated Reverse Logistics Management Portal

Task 1: Problem Definition and Scope | Task 2: Agile Planning and DevOps Workflow

Item	Details
Student name	Soham Kholapure
Roll number	23102A0064
Class / Division	BE CMPN A
Project serial number	60
Course	DevOps (Mini Project)
Project topic	Automated Reverse Logistics Management Portal
Tasks covered	Task 1 and Task 2 (of 15)

Contents of this document

Task	Required deliverable	Where in this document
Task 1	Problem statement	Section 1.2
Task 1	Stakeholder list	Section 1.4
Task 1	Objectives	Section 1.5 (and success criteria in 1.6)
Task 1	Constraints	Section 1.7
Task 1	Approved MVP scope	Sections 1.9 to 1.11
Task 2	Backlog (user stories and product backlog)	Sections 2.2 and 2.3
Task 2	Task board and sprint plan	Sections 2.4 and 2.5
Task 2	Definition of Done	Section 2.6
Task 2	DevOps workflow diagram	Section 2.7

Task 1: Problem Definition and Scope

1.1 Background and Real-Time Need

Reverse logistics is the movement of goods from the customer back to the seller or manufacturer for return, replacement, repair, refurbishment, resale, recycling or disposal. As online shopping has grown, handling returns has become a daily operational activity for e-commerce sellers, retailers and distributors. It directly affects customer satisfaction, refund timelines and inventory accuracy.

In many small and mid-size businesses, returns are still handled through phone calls, e-mails, messaging applications and spreadsheets. No single place exists where a customer, a manager and a warehouse employee can see the same, current information about a return. This gap creates a real need for a portal that records every return, moves it through a defined workflow and gives managers an up-to-date summary.

The need for the portal can be summarised as follows:

- **Visibility:** everyone involved should see the current status of each return.
- **Control:** returns should follow a fixed approval and inspection process, with each step done by the correct role.
- **Speed:** faster decisions on refund, restock or disposal reduce losses and customer waiting time.
- **Accuracy:** a recorded inspection outcome keeps inventory and refund data correct.

1.2 Problem Statement

> Businesses that manage product returns manually through e-mail, calls and spreadsheets cannot track the status of returned items, lose visibility of what is arriving at the warehouse, and delay refund and restocking decisions. The **Automated Reverse Logistics Management Portal** is a web application in which customers raise return requests and managers and warehouse staff process them through a role-based status workflow: request, approval, pickup, receipt, inspection and final outcome (refund, restock or disposal), supported by a live summary dashboard. The project also builds a complete DevOps pipeline (Git, Jenkins, Selenium, Docker and Ansible) so that the portal is built, tested, packaged and deployed automatically.

1.3 Existing Pain Points

ID	Pain point in the current manual process	Impact	Addressed by
P1	Returns are recorded in e-mails and spreadsheets	Duplicate or lost entries; no single source of truth	F2, F3
P2	Customers cannot see the status of their return	Repeated follow-up calls; poor customer experience	F3, F6
P3	No fixed approval workflow	Inconsistent decisions and delays	F6
P4	Warehouse does not know which returns are coming	Poor planning of receiving and inspection	F3, F6
P5	Inspection results are not recorded properly	Items wrongly restocked or disposed; stock mismatch	F4, F6
P6	Refund is not linked to the inspection result	Delayed or incorrect refunds	F6

ID	Pain point in the current manual process	Impact	Addressed by
P7	No summary of pending and completed returns	Managers cannot monitor progress	F7
P8	Manual deployment of software changes (for the portal itself)	Slow, error-prone releases	DevOps pipeline

1.4 Target Users and Stakeholder List

Stakeholder	Type	Role in the return process	Main need
Customer	Primary user	Raises a return request and follows its progress	Easy request form and clear status
Return Manager	Primary user (admin)	Approves or rejects returns, schedules pickup, records refunds, monitors the dashboard	Control, visibility and summary counts
Warehouse Staff	Primary user	Marks items received, records inspection, decides restock or disposal	List of expected returns and a simple way to record outcomes
Finance Team	Secondary	Uses refund information for payment and accounts	Accurate refund amounts and status
Courier / Pickup Partner	Secondary (outside the system)	Collects the item on the scheduled date	Clear pickup date from the manager
Course Instructor	Project stakeholder	Evaluates the project and DevOps practices	Working MVP, complete evidence and documentation
Developer / DevOps Engineer (Soham Kholapure)	Project team	Designs, builds, tests, containerises and deploys the portal	Clear scope and an automated delivery pipeline

1.5 Objectives

1. Provide a web portal to **create, view, update and search** return requests.
2. Enforce a **role-based status workflow** so that each step is done by the correct role.
3. Provide a **summary dashboard** with counts per status and total refund amount.
4. Use **Git and GitHub** with branching, pull requests, conflict resolution and version tags.
5. Automate build and test using **Jenkins and Selenium**, with failed tests blocking deployment.
6. Package the portal with **Docker**, publish versioned images and deploy containers automatically.
7. Use **Ansible** to provision a clean target node repeatably, with health check and rollback.
8. Produce complete documentation, a troubleshooting guide and a final demonstration.

1.6 Measurable Success Criteria

ID	Criterion	How it is measured	Target
SC1	A customer can create a return request quickly	Timed manual test of the create form	60 seconds or less
SC2	Every return always has a valid status	Check stored records against the defined status list	100% of records

ID	Criterion	How it is measured	Target
SC3	Dashboard counts are correct	Compare dashboard figures with stored records	100% match
SC4	Search returns correct results	Search by order number, customer name and status	All matching records; response under 2 seconds
SC5	Role restrictions are enforced	Test that a role cannot perform actions of another role	100% of unauthorised actions blocked
SC6	Automated UI tests pass before deployment	Selenium suite run by Jenkins	3 to 5 journeys, all pass
SC7	Fast automated delivery	Time from commit to running container	10 minutes or less
SC8	Reliable provisioning	Run the Ansible playbook a second time	Zero changes (changed=0)
SC9	Healthy deployment and recovery	Health check after deployment; rollback to previous release	Status UP; rollback successful

1.7 Constraints

Category	Constraint
Time	The work must follow the 15-task course schedule; one task per deliverable.
Team	Single developer (Soham Kholapure).
Cost	Only free and open-source tools or free tiers (Spring Boot, Maven, Git/GitHub, Jenkins, Docker and Docker Hub, Ansible).
Technology	Java 17, Spring Boot 3, Maven, H2 database, Tomcat 10.1, Ubuntu (WSL2 or virtual machine).
Infrastructure	Everything runs on a personal laptop or virtual machines. Jenkins uses SCM polling because a local server cannot receive GitHub webhooks.
Ports	Jenkins 8080; application in development 8081; standalone Tomcat 8082; Docker container 8083.
Security	Simple role-based login with predefined users; not production-grade security.
Scope	No payment gateway, courier API, e-mail or SMS integration.

1.8 Assumptions

- Users access the portal through a web browser.
- Order details are entered manually by the user; there is no connection to an order management system.
- There is a single warehouse and each return request covers one product.
- The manager schedules the pickup; the courier is not part of the software.
- The refund amount is recorded in the portal; the actual payment is made outside the system.

1.9 MVP Scope (Frozen)

The Minimum Viable Product (MVP) covers create, view, update and search of return records, a role-based status workflow and a summary dashboard, together with the full DevOps pipeline.

In-scope features

ID	Feature	Description
F1	Login and role-based access	Login for three roles: Customer, Return Manager, Warehouse Staff. Pages and actions depend on the role.
F2	Create return request	Customer submits order number, name, product, quantity and reason. Status starts as REQUESTED.
F3	View returns	List and detail pages. Customers see their own records; manager and warehouse see all.
F4	Update return request	Edit allowed fields according to role and current status.
F5	Search and filter	Search by order number or customer name; filter by status.
F6	Role-based status workflow	Controlled movement of a return through the statuses in the table below.
F7	Summary dashboard	Total returns, count per status and total refund amount.
F8	Health endpoint	/actuator/health to confirm the deployed application is running.

Status workflow and permitted roles

From status	To status	Action	Allowed role
(new)	REQUESTED	Create return request	Customer
REQUESTED	APPROVED	Approve	Return Manager
REQUESTED	REJECTED	Reject (remarks required)	Return Manager
APPROVED	PICKUP_SCHEDULED	Schedule pickup date	Return Manager
PICKUP_SCHEDULED	RECEIVED	Mark item received	Warehouse Staff
RECEIVED	INSPECTED	Record inspection (remarks required)	Warehouse Staff
INSPECTED	REFUNDED	Record refund (amount required)	Return Manager
INSPECTED	RESTOCKED	Return item to stock	Warehouse Staff
INSPECTED	DISPOSED	Dispose of item	Warehouse Staff

REJECTED, REFUNDED, RESTOCKED and DISPOSED are final statuses; a record in a final status cannot be changed.

Role permission matrix

Function	Customer	Return Manager	Warehouse Staff
Log in	Yes	Yes	Yes
Create return request	Yes	No	No
View returns	Own records only	All records	All records
Update details	Own record, only while REQUESTED	Remarks and refund amount	Inspection remarks
Search and filter	Yes (own records)	Yes	Yes
Approve, reject, schedule pickup	No	Yes	No
Mark received and inspected	No	No	Yes
Record final outcome	No	Refund	Restock or dispose
View dashboard	No	Yes	Yes

Out of scope (future enhancements)

Payment gateway, real courier API integration, e-mail or SMS notifications, barcode or QR return labels, multiple warehouses, password encryption with Spring Security, production database (MySQL or PostgreSQL), mobile application and report export.

1.10 The 15-Task Plan for the MVP

The MVP is delivered through the following 15 tasks. Each task produces one deliverable and is tracked as one issue on the project board.

Task	Title	Main deliverable
1	Problem Definition and Scope	Problem statement, stakeholders, objectives, constraints, MVP scope
2	Agile Planning and DevOps Workflow	Backlog, task board, sprint plan, Definition of Done, workflow diagram
3	Requirements, Architecture and Technology Setup	SRS summary, use-case and architecture diagrams, data model, working local setup
4	Git and GitHub Repository Initialization	Repository, README, initial commits, issues, branch policy
5	Feature Development with Branching	Feature 1 on a branch, pull request, review, merge
6	MVP Completion and Git Collaboration	Functional MVP, resolved merge conflict, tagged version, updated backlog
7	Jenkins Installation and Continuous Integration Job	Jenkins job, build log, trigger evidence, archived artefact
8	Pipeline as Code and Server Deployment	Jenkinsfile, pipeline run, deployed application, parameterised setting
9	Selenium Test Design and Local Execution	Test plan, Selenium scripts, local report, failure screenshots
10	Continuous Testing in Jenkins	Jenkins test report, failed run, defect fix, successful rerun
11	Docker Image and Container Lifecycle	Dockerfile, image details, command log, running container

Task	Title	Main deliverable
12	Jenkins-Docker Continuous Deployment	Versioned image, registry evidence, commit-to-container pipeline
13	Configuration Management Script	Configuration specification, Ansible playbook, first execution log
14	Automated Provisioning and Reliability Validation	Provisioned node, idempotency, health check, rollback
15	Final End-to-End Release, Documentation and Viva	Final repository, live demo, report, presentation, viva

1.11 MVP Scope Approval

The scope in Sections 1.9 and 1.10 is frozen. Any new idea raised after approval is added to the product backlog as a future enhancement and is not part of this MVP.

	Name	Signature	Date
Prepared by	Soham Kholapure (Roll No. 23102A0064)		
Approved by (Course Faculty)			

Task 2: Agile Planning and DevOps Workflow

2.1 Agile Approach

The project follows **Scrum** for planning (three sprints) combined with a **Kanban board** for daily tracking. Because there is one developer, the Scrum roles are combined: Soham Kholapure acts as Product Owner, Scrum Master and Developer, and the course instructor acts as the stakeholder who reviews each sprint. Work is tracked on a GitHub Project board; each of the 15 tasks is one GitHub issue.

Scrum ceremonies adapted for this project:

Ceremony	When	Purpose in this project
Sprint planning	Start of each sprint	Select the tasks and backlog items for the sprint and confirm acceptance criteria.
Daily check-in	Each working day (5 minutes, self)	Note what was done, what is next and what is blocked, as a comment on the GitHub issue.
Sprint review	End of each sprint	Demonstrate the working result and the evidence to the faculty.
Retrospective	End of each sprint	Record what went well and what to improve in the next sprint.

2.2 User Stories and Acceptance Criteria

Priority uses the MoSCoW method (all MVP stories are Must). Points are relative effort estimates using the Fibonacci scale.

ID	User story	Acceptance criteria	Pri.	Pts
US-01	As a user (customer, manager or warehouse staff), I want to log in with my username and password so that I see only the functions allowed for my role.	AC1: Valid credentials open the portal home page for that role. AC2: Invalid credentials show an error message and no session is created. AC3: Pages and actions not allowed for the role are blocked with an "Access denied" message.	Must	3
US-02	As a customer, I want to raise a return request for a purchased item so that the seller can process my return.	AC1: Order number, customer name, product name, quantity (1 or more) and reason are mandatory; invalid input shows validation messages. AC2: On save the request gets a unique ID, status REQUESTED and creation date. AC3: A confirmation message and the new record are shown.	Must	5
US-03	As a user, I want to see a list of return requests so that I can quickly find the ones I need to act on.	AC1: The list shows ID, order number, product, status and date. AC2: A customer sees only own requests; manager and warehouse see all. AC3: Newest requests appear first.	Must	3
US-04	As a user, I want to open a return request to see its full details and current status.	AC1: The details page shows all fields, status and remarks. AC2: Only the actions allowed for my role and the current status are shown as buttons.	Must	2

ID	User story	Acceptance criteria	Pri.	Pts
US-05	As a user, I want to update a return request so that wrong or missing information can be corrected.	AC1: A customer can edit reason and quantity only while the status is REQUESTED. AC2: A manager can edit remarks and refund amount; warehouse staff can edit inspection remarks. AC3: Updated values are saved and visible immediately.	Must	3
US-06	As a user, I want to search returns by order number, customer name or status so that I can find a record quickly.	AC1: Search by order number or customer name returns all matching records (partial and case-insensitive). AC2: A status filter returns only records in that status. AC3: A "No records found" message appears when nothing matches.	Must	3
US-07	As a return manager, I want to approve or reject a return request so that only valid returns proceed.	AC1: Approve and Reject are available only for REQUESTED returns and only to the manager. AC2: Rejection requires remarks. AC3: Status changes to APPROVED or REJECTED and is shown in the list.	Must	3
US-08	As a return manager, I want to schedule a pickup for an approved return so that the item is collected.	AC1: Available only for APPROVED returns. AC2: Pickup date is mandatory and cannot be in the past. AC3: Status changes to PICKUP_SCHEDULED.	Must	2
US-09	As warehouse staff, I want to mark a return as received and then inspected so that the condition of the item is recorded.	AC1: Received is available only for PICKUP_SCHEDULED returns; Inspected only for RECEIVED returns. AC2: Inspection remarks are mandatory when marking INSPECTED.	Must	3
US-10	As a manager or warehouse staff member, I want to record the final outcome of an inspected return so that the item is refunded, restocked or disposed.	AC1: Only INSPECTED returns accept an outcome: manager records REFUNDED (refund amount required); warehouse records RESTOCKED or DISPOSED. AC2: Returns in a final status cannot be changed further.	Must	3
US-11	As a return manager, I want a dashboard with summary counts so that I can monitor progress.	AC1: Shows total returns and the count for each status. AC2: Shows the total refund amount. AC3: Counts match the stored records (verified by an automated test).	Must	5
US-12	As a developer, I want automated build, test and deployment so that every change is delivered quickly and reliably.	AC1: Every commit to the repository triggers a Jenkins build. AC2: Failed tests stop the pipeline before deployment. AC3: A successful run produces a versioned Docker image that is deployed as a container.	Must	8

2.3 Product Backlog

Items are listed in priority order. B01 to B19 form the MVP; B20 onwards are future enhancements that are outside the frozen scope.

ID	Backlog item	Stories	Task	Priority	Sprint
B01	Problem definition and MVP scope	-	1	Must	1
B02	Backlog, task board, sprint plan, Definition of Done, workflow diagram	-	2	Must	1
B03	SRS summary, architecture, technology setup and local environment	-	3	Must	1
B04	GitHub repository, README, issue templates, branch policy	-	4	Must	1
B05	Create return request (feature 1) with pull request and review	US-02	5	Must	1
B06	View return list and details	US-03, US-04	6	Must	2
B07	Update and search return requests	US-05, US-06	6	Must	2
B08	Login and role-based access	US-01	6	Must	2
B09	Role-based status workflow	US-07 to US-10	6	Must	2
B10	Summary dashboard; conflict resolution; tag v1.0.0	US-11	6	Must	2
B11	Jenkins installation and CI build job	US-12	7	Must	2
B12	Jenkinsfile and deployment to Tomcat	US-12	8	Must	2
B13	Selenium test plan and scripts for critical journeys	US-01 to US-11	9	Must	2
B14	Selenium tests in Jenkins as a quality gate	US-12	10	Must	2
B15	Dockerfile and container lifecycle	US-12	11	Must	3
B16	Jenkins-Docker continuous deployment	US-12	12	Must	3
B17	Ansible configuration specification and playbook	US-12	13	Must	3
B18	Provisioning, idempotency, health check and rollback	US-12	14	Must	3
B19	Final release, documentation, demo and viva	All	15	Must	3
B20	Spring Security with encrypted passwords	-	-	Future	-
B21	Production database (MySQL or PostgreSQL)	-	-	Future	-
B22	E-mail or SMS notifications on status change	-	-	Future	-
B23	QR-code return labels and courier API integration	-	-	Future	-
B24	Monitoring with Prometheus and Grafana; Kubernetes deployment	-	-	Future	-

2.4 Sprint Plan (3 Sprints, 15 Tasks)

Sprint	Sprint goal	Tasks and backlog	Planned deliverables
Sprint 1 (about 2 to 3 weeks)	Plan, set up and deliver the first feature	Tasks 1 to 5 B01 to B05	Approved scope; backlog and board; SRS and architecture with local setup; GitHub repository; feature 1 merged through a pull request
Sprint 2 (about 2 to 3 weeks)	Complete the MVP and automate build and test	Tasks 6 to 10 B06 to B14	Functional MVP tagged v1.0.0; Jenkins CI job; Jenkinsfile with Tomcat deployment; Selenium suite; test quality gate in Jenkins
Sprint 3 (about 2 to 3 weeks)	Containerise, provision and release	Tasks 11 to 15 B15 to B19	Docker image and lifecycle; Jenkins-Docker deployment; Ansible playbook; provisioned node with rollback; final report, demo and viva

2.5 Kanban Task Board (15-Task Plan)

Board columns: Backlog, To Do (current sprint), In Progress, Review and Done. The work-in-progress limit for In Progress is 2 cards. The snapshot below shows the board at the start of Sprint 1; the live board is maintained as a GitHub Project.

Backlog (Sprints 2 and 3)	To Do (Sprint 1)	In Progress (limit 2)	Review	Done
T6 MVP Completion and Git Collaboration T7 Jenkins CI Job T8 Jenkinsfile and Tomcat Deployment T9 Selenium Tests T10 Continuous Testing in Jenkins T11 Docker Image and Lifecycle T12 Jenkins-Docker Deployment T13 Ansible Configuration T14 Provisioning and Rollback T15 Final Release and Viva	T3 Requirements, Architecture and Setup T4 GitHub Repository T5 Feature 1 with Branching	T2 Agile Planning and DevOps Workflow	(empty)	T1 Problem Definition and Scope

Card workflow: a card moves from Backlog to To Do at sprint planning, to In Progress when work starts, to Review when a pull request or evidence is ready, and to Done only when the Definition of Done (Section 2.6) is met.

2.6 Definition of Done

A task or user story is Done only when all applicable criteria below are met.

#	Criterion	Applies from
1	Code is committed to a feature branch with meaningful commit messages and pushed to GitHub.	Task 4
2	A pull request is raised, reviewed and merged into the develop branch.	Task 5
3	The application builds successfully with Maven (mvn clean package) with no errors.	Task 3
4	All acceptance criteria of the story or task are demonstrated.	All tasks
5	Automated Selenium tests pass locally.	Task 9
6	The Jenkins build or pipeline run is green; failed tests block deployment.	Task 7 (build), Task 10 (tests)
7	The application is deployed and verified on the target (Tomcat or container) with a health check.	Task 8, Task 12
8	No open blocking defects; known issues are logged as GitHub issues.	All tasks
9	Evidence (screenshots, logs, command output) is saved in the docs folder of the repository.	All tasks
10	README and documentation are updated.	All tasks
11	The board card is moved to Done and the GitHub issue is closed.	All tasks

2.7 DevOps Lifecycle and Workflow

DevOps joins development and operations in one continuous loop. In this project the loop has eight stages, from planning to monitoring, and the feedback from monitoring returns to planning as new backlog items.

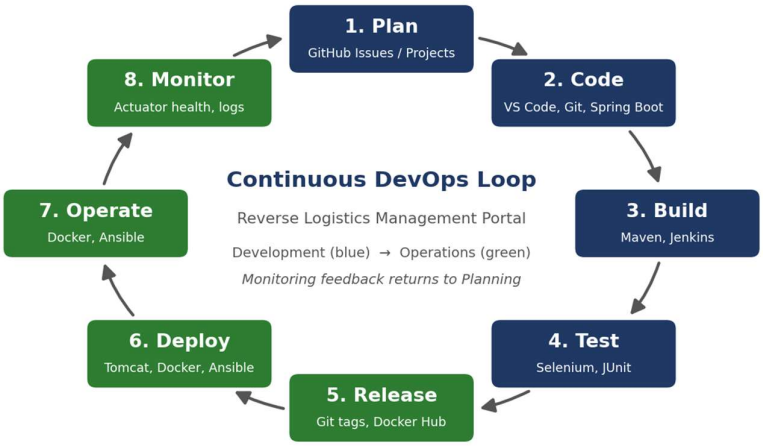


Figure 1: DevOps lifecycle for the Reverse Logistics Management Portal

Stage	What happens	Tools	Tasks
1. Plan	Define scope, stories, backlog and sprint plan	GitHub Issues and Projects, draw.io	1, 2
2. Code	Develop features on branches; commit; review through pull requests	VS Code, Git, GitHub, Spring Boot, Maven	3 to 6
3. Build	Compile and package the application into a WAR file on every commit	Maven, Jenkins	7, 8
4. Test	Run Selenium UI tests; a failed test stops the pipeline	Selenium WebDriver, JUnit, Jenkins	9, 10
5. Release	Tag the version; publish a versioned Docker image	Git tags, Docker, Docker Hub	6, 11, 12
6. Deploy	Deploy the WAR to Tomcat or run the container on the target node	Tomcat, Docker, Ansible	8, 12, 14
7. Operate	Keep the target environment consistent; roll back when needed	Docker, Ansible	13, 14
8. Monitor	Check application health and logs; feed findings back to planning	Spring Boot Actuator, Jenkins console, Docker logs	14, 15

Commit-to-container workflow

Figure 2 shows how a code change travels automatically from the developer to the running application.

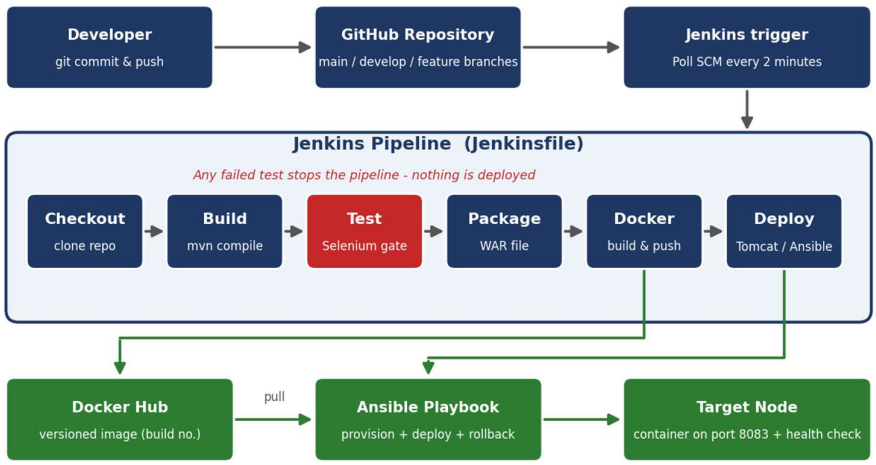


Figure 2: Automated commit-to-container workflow

1. The developer commits and pushes code to GitHub.
2. Jenkins detects the change by polling the repository every two minutes and starts the pipeline defined in the Jenkinsfile.
3. The pipeline checks out the code, builds it with Maven and runs the Selenium tests.
4. If any test fails, the pipeline stops and nothing is deployed.
5. If all tests pass, the application is packaged as a WAR file and a versioned Docker image is built and pushed to Docker Hub.
6. The Ansible playbook pulls the image and runs the container on the target node.
7. A health check on /actuator/health confirms that the application is running (status UP).
8. If a release is faulty, the playbook is run again with the previous image version to roll back.